

3.4 while Loops

[This section corresponds to half of K&R Sec. 3.5]

Loops generally consist of two parts: one or more *control expressions* which (not surprisingly) control the execution of the loop, and the *body*, which is the statement or set of statements which is executed over and over.

The most basic *loop* in C is the `while` loop. A `while` loop has one control expression, and executes as long as that expression is true. This example repeatedly doubles the number 2 (2, 4, 8, 16, ...) and prints the resulting numbers as long as they are less than 1000:

```
int x = 2;

while(x < 1000)
{
    printf("%d\n", x);
    x = x * 2;
}
```

(Once again, we've used braces `{ }` to enclose the group of statements which are to be executed together as the body of the loop.)

The general syntax of a `while` loop is

```
while( expression )
    statement
```

A `while` loop starts out like an `if` statement: if the condition expressed by the *expression* is true, the *statement* is executed. However, after executing the statement, the condition is tested again, and if it's still true, the statement is executed again. (Presumably, the condition depends on some value which is changed in the body of the loop.) As long as the condition remains true, the body of the loop is executed over and over again. (If the condition is false right at the start, the body of the loop is not executed at all.)

As another example, if you wanted to print a number of blank lines, with the variable `n` holding the number of blank lines to be printed, you might use code like this:

```
while(n > 0)
{
    printf("\n");
    n = n - 1;
}
```

After the loop finishes (when control "falls out" of it, due to the condition being false), `n` will have the value 0.

You use a `while` loop when you have a statement or group of statements which may have to be executed a number of times to complete their task. The controlling expression represents the condition "the loop is not done" or "there's more work to do." As long as the expression is true, the body of the loop is executed; presumably, it makes at least some progress at its task. When the expression becomes false, the task is done, and the rest of the program (beyond the loop) can proceed. When we think about a loop in this way, we can see an additional important property: if the expression evaluates to "false" before the very first trip through the loop, we make *zero* trips through the loop. In other words, if the task is already done (if there's no work to do) the body of the

loop is not executed at all. (It's always a good idea to think about the ``boundary conditions" in a piece of code, and to make sure that the code will work correctly when there is no work to do, or when there is a trivial task to do, such as sorting an array of one number. Experience has shown that bugs at boundary conditions are quite common.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)